Лабораторная №2. План подготовки

Тема: параллельные и последовательные области. Этот материал предназначен для устной защиты: не только назвать директиву, но и объяснить, кто исполняет код, какие данные меняются, где нужна синхронизация и почему результат именно такой. Актуален для текущего Lab2.cpp на 29 октября 2026 года.

Короткий ответ на вопрос «что сделано?»

«Я исследовал создание команды OpenMP, условное распараллеливание, число доступных логических процессоров и три уровня вложенности. Сравнил single и master. В первом задании запрашиваю всё большие команды в отдельных процессах, а при обнаружении отказа уточняю и повторно проверяю границу. Экспериментальная граница - 8192 нити: запросы 8193 и 10000 создают только 8192. Соседние точки границы проверены повторно».

Раздел	Что нужно уметь объяснить
Основы	Нить, процесс, ядро, логический процессор; fork-join; общий адресный простор
API и настройки	Различия get_num_threads, get_max_threads, get_num_procs; num_threads и dynamic
Задания 1-3	Как искать предел; зачем if; почему число нитей может быть больше числа процессоров
Задание 4	Откуда берутся 8 выполнений на третьем уровне; что меняет запрет вложенности
Задания 5-7	Кто исполняет single/master; где барьер; почему нить 1 не является мастером команды
Разбор кода	reduction, atomic, critical, private/firstprivate; меню и служебные процессы

Как показать работу преподавателю

Открыть ParallelProgramming.sln, собрать x64 Debug или Release. В главном меню выбрать **2**; пункты **1-7** показывают задания, **8** запускает всё. Сначала назвать ожидаемый результат, потом запустить и объяснить вывод. Не обещать фиксированный порядок строк от разных нитей.

Важно: прежняя проверка до 128 была недостаточной для определения максимума. Новая программа различает найденную границу и защитный предел. В проведённом эксперименте предел поиска равен 10000, а наблюдаемая граница полного выполнения запроса - 8192. Это результат для текущей сборки и runtime, не число физических ядер.

1. Что такое параллельная область

Процесс, нить и процессор - не одно и то же

Процесс имеет свой адресный простор и ресурсы. **Нити** одного процесса исполняют команды и в основном разделяют память процесса, но имеют собственные стеки и контекст исполнения. **Физическое ядро** - часть процессора, исполняющая инструкции; **логический процессор** - аппаратный контекст исполнения, который видит ОС. На одном ядре может быть несколько логических процессоров.

OpenMP - модель и API параллельного программирования с общей памятью: директивы компилятора, функции библиотеки и переменные среды. В проекте включён /openmp. Без поддержки компилятором одного подключения omp.h недостаточно. Это не MPI: программа не создаёт независимые вычислительные процессы с обменом сообщениями вместо общей памяти.

Модель fork-join

```
Последовательный код: начальная нить
parallel: команда N нитей, номера 0 ... N-1
Конец parallel: ожидание завершения всех участников
Следующий последовательный код: породившая нить
```

При входе в parallel породившая нить участвует в новой команде как нить 0, а остальные участники исполняют тот же блок. В команде из четырёх нитей блок выполняется четыре раза. Размер команды не меняется посреди этой области. На выходе есть неявный барьер: следующий последовательный оператор не начнётся раньше завершения участников.

Это логическая модель. Реализация может держать рабочие потоки в пуле и повторно использовать их, а не создавать и физически уничтожать каждый раз. Поэтому формулировка «нити исчезают на выходе» означает завершение их участия в области, а не обязательное уничтожение потоков ОС.

parallel не делит обычный цикл автоматически

Если поместить обычный for внутрь parallel без omp for, каждая нить выполнит весь цикл. Директива for распределяет итерации между участниками уже созданной команды; parallel for объединяет создание команды и распределение. Сам по себе omp for новую команду не создаёт.

Одновременный запуск нитей не гарантирует ускорение. Если нитей больше аппаратных контекстов, ОС переключает их; часть времени уходит на ожидание и синхронизацию. Строки вывода отражают порядок получения доступа к консоли, а не точную шкалу параллельного исполнения.

2. Функции и настройки OpenMP

Вызов	Что означает и что НЕ означает
omp_get_num_threads()	Размер текущей команды. В обычном последовательном контексте - 1. Измерять фактическую команду нужно внутри области.
omp_get_thread_num()	Номер в текущей команде от 0 до N-1. Это не глобальный ID потока ОС и не номер ядра.
omp_get_num_procs()	Количество логических процессоров, доступных программе по данным runtime; не предел числа создаваемых нитей.
omp_get_max_threads()	Настройка размера следующей команды без своего num_threads в данном контексте. Не экспериментальный аппаратный максимум.
omp_set_num_threads(N)	Меняет настройку следующих областей в контексте вызова, но не текущую команду и не текст переменной среды.
omp_set_dynamic(0)	Запрещает runtime динамически уменьшать запрашиваемые команды по этому механизму; не отменяет прочие ограничения.
omp_in_parallel()	Ненулевое значение в активной параллельной области. При сериализованной внешней области if(false) - 0.
omp_set_nested(1)	Разрешает вложенный параллелизм в используемом API. В современных версиях есть max_active_levels.

Как выбирается запрашиваемый размер

Для конкретной parallel-области явный num_threads(N) задаёт запрос этой области. Если его нет, используется текущая настройка, которую можно изменить omp_set_num_threads(). Первоначально она зависит от OMP_NUM_THREADS и реализации. Например, set_num_threads(2), затем область num_threads(3), затем область без опции дадут запросы 3 и 2, а не два раза 3.

Запрос и фактический результат различаются: учитываются if, разрешённая вложенность, ограничения реализации и ресурсы. В новых runtime есть OMP_THREAD_LIMIT и OMP_MAX_ACTIVE_LEVELS, но не следует обещать поддержку современных настроек обычным режимом MSVC /openmp.

В PowerShell перед запуском можно задать $env:OMP_NUM_THREADS="4" и $env:OMP_DYNAMIC="FALSE". runLab2() сохраняет прежнее dynamic, временно выключает его и восстанавливает на выходе. Изменения не должны неожиданно влиять на следующие лабораторные.

3. Задание 1: экспериментальный предел

Что именно измеряется

Ищется наибольший запрос N, при котором отдельная тестовая команда действительно выполняется ровно N нитями. Это предел для конкретного исполняемого файла, runtime, настроек и текущего состояния ресурсов, а не вечная характеристика процессора. На другом устройстве и даже при другой нагрузке результат может измениться.

Алгоритм showThreadCount()

1. Запрашиваются 1, 2, 4, 8 и далее степени двойки. В каждой пробе отключён dynamic, фактическая команда измеряется внутри parallel. Пока actual == requested, нижняя подтверждённая граница растёт.

2. Если проба завершилась отказом runtime или создала меньшую команду, появляется неуспешная верхняя граница. Между последним успехом и отказом проверяется середина интервала. Это двоичный поиск: успешная середина поднимает lower, неуспешная опускает upper.

3. Когда upper = lower + 1, обе соседние точки проверяются ещё три раза. Только стабильный успех lower и стабильный неполный запрос upper позволяют вывести «экспериментальный максимум полного выполнения запроса». Тайм-аут и ошибка запуска помечаются как неопределённый результат, а не доказанный отказ создания команды.

Результат на этом компьютере

Показатель	Измерение 29 октября 2026
Доступные логические процессоры	12
Настройка следующей команды в обычном запуске	12
Полностью успешные запросы	Степени двойки от 1 до 8192 включительно
Запрос 8193	Создано 8192; повторено на границе три раза
Запрос 10000	Создано 8192, а не 10000
Экспериментальная граница	8192 нити для текущей сборки и runtime

После отдельных согласованных проб 8192 и 10000 двоичный поиск проверил промежуточные запросы до соседней пары 8192/8193. Затем обе точки проверены ещё три раза: 8192 всегда выполнен полностью, 8193 всегда уменьшен до 8192. Защитный предел поиска 10000 не выдаётся за измеренный максимум. Нет оснований считать 8192 универсальным пределом Windows, всех runtime или всех устройств.

4. Как устроена безопасная проба

Зачем отдельный процесс

Некоторые runtime при невозможности создать нити аварийно завершают процесс. Обычный try/catch вокруг parallel не гарантирует перехват такого отказа. Поэтому probeThreadCount() запускает этот же exe в служебном режиме --lab2-thread-probe N. Рабочая программа остаётся родителем и получает результат завершившейся пробы.

```
int actual = 0;
omp_set_dynamic(0);
#pragma omp parallel num_threads(requested) shared(actual)
{
#pragma omp single
    actual = omp_get_num_threads();
}
```

Это тело measureLab2Team(). single нужен, чтобы только одна нить записала общий actual: даже одинаковая одновременная запись несколькими нитями не является корректной заменой синхронизации. Конец области гарантирует завершение команды до возврата результата.

Служебный протокол и ограничения

main() проверяет число из аргумента через from_chars: допускаются только целые значения 1-10000 без лишних символов. Результат передаётся как код завершения с маркером lab2ProbeExitBase + actual. Маркер не даёт спутать обычный код ошибки вроде 3 с командой из трёх нитей. Это внутренний протокол Windows, а не обычный режим работы меню.

GetModuleFileNameW определяет путь текущего exe; CreateProcessW запускает только этот файл. Путь заключён в кавычки, поэтому пробелы допустимы. WaitForSingleObject ждёт до 10 секунд. Если процесс не завершился, родитель прекращает только свою пробу и не объявляет её ресурсным пределом. HANDLE закрываются, чтобы не накапливать утечки. CREATE_NO_WINDOW скрывает служебное окно.

Порог общего времени проверяется после проб: при превышении 90 секунд поиск прекращается. Это не строгий секундомер всего задания: последняя уже начавшаяся проба и повторная пара могут добавить время. Изоляция защищает основную программу от аварии ребёнка, но сама по себе не защищает всю ОС от чрезмерной нагрузки, поэтому есть предел запроса.

Что сказать про точность

Двоичный поиск предполагает практически монотонную границу при достаточно стабильных ресурсах: большая команда не должна становиться легче меньшей. Три повтора проверяют устойчивость соседних точек, но не доказывают все мыслимые запросы во всех состояниях системы. На защите корректно говорить о наблюдаемой экспериментальной границе при оговорённых условиях.

5. Задание 2: условие if и сумма

Когда распараллеливание оправдано

У создания команды, распределения работы и барьеров есть накладные расходы. На небольшом объёме данных они могут стоить дороже самих вычислений. if позволяет выполнить один и тот же блок одной нитью для маленького входа и командой для большого. Порог выбирают измерением производительности, а не потому, что существует универсальное число 100000.

```
#pragma omp parallel if(size >= 100000) num_threads(4) \
    shared(actual) reduction(+:sum)
{
#pragma omp single
    actual = omp_get_num_threads();
#pragma omp for
    for (int i = 1; i <= size; i++)
    {
        sum += i;
    }
}
```

showConditionalParallel() запускает размеры 10 и 1 000 000. При ложном условии блок **не пропускается**: он выполняется как сериализованная parallel-область командой из одной нити. При истинном условии запрашиваются четыре нити. Поэтому маленький вход даёт 1 нить и сумму 55, большой - 4 нити и сумму 500000500000 в проведённом запуске.

Почему сумма правильная

omp for распределяет итерации, чтобы каждое слагаемое учитывалось один раз. reduction(+:sum) создаёт приватные аккумуляторы, инициализированные нулём, и объединяет их сложением с исходной sum. Без reduction общий sum += i был бы гонкой данных. Если убрать for, каждая нить просуммирует весь диапазон, а reduction объединит повторно вычисленные суммы.

Контрольная формула: N * (N + 1) / 2. Тип long long нужен для 500000500000; int на платформе проекта слишком мал. Приведение static_cast<long long>(size) выполняется до умножения, иначе переполниться могло бы уже промежуточное выражение.

В этом задании проверяется корректность выбора режима и суммы, а не выигрыш во времени. Чтобы заявлять ускорение, нужны сопоставимые повторные замеры omp_get_wtime(), Release-сборка и отсутствие консольного вывода в измеряемом участке.

6. Задание 3: занять процессоры

Что делает showAllProcessors()

omp_get_num_procs() вернул 12. Значение processors используется в num_threads(processors). В проведённом запуске 12 участников вывели свои номера от 0 до 11 и размер команды 12. critical(outputLab2) не даёт смешаться защищённым операциям вывода разных нитей.

```
int processors = omp_get_num_procs();
#pragma omp parallel num_threads(processors)
{
#pragma omp critical(outputLab2)
    cout << omp_get_thread_num() << " / "
         << omp_get_num_threads() << '\n';
}
```

Запросить столько же нитей, сколько логических процессоров, и жёстко закрепить по одной нити на каждом - разные действия. Этот пример создаёт команду нужного размера, но не задаёт привязку к процессорам. ОС может переносить нити между доступными контекстами. Короткий вывод не доказывает, что все процессоры были непрерывно заняты полезной работой.

Привязка, перегрузка и производительность

Для строгой привязки нужен поддерживаемый механизм affinity: например, OMP_PROC_BIND и OMP_PLACES в подходящем современном runtime или специальные средства ОС. Нельзя просто добавить современную переменную среды и считать, что старый MSVC /openmp её обязательно исполнил.

8192 программные нити при 12 логических процессорах - oversubscription, то есть переподписка аппаратных ресурсов. Это допустимо как эксперимент создания, но не означает 8192 одновременных аппаратных исполнителя. Для вычислительной нагрузки лишние нити могут ухудшить время из-за переключений, ожидания и конкуренции за память.

Даже команда в число логических процессоров не всегда оптимальна: важны физические ядра, память, характер вычислений, фоновые процессы и размер входа. Для большого объёма ожидания внешних операций иногда полезно больше нитей; для CPU-нагрузки это не общее правило.

Если спросят про закон Амдала

Если доля строго последовательной работы равна s, идеальное ускорение на p исполнителях ограничено S(p) = 1 / (s + (1 - s) / p), ещё без учёта накладных расходов. При s = 0.1 даже бесконечное число исполнителей не даст ускорение больше 10. Поэтому максимальное число создаваемых нитей и максимальная производительность не равны.

7. Задание 4: три уровня вложенности

Как работает рекурсивная nestedRegion()

Каждый вызов создаёт parallel num_threads(2). На уровнях 1 и 2 каждая нить вызывает nestedRegion(level + 1, path, leafCount). На уровне 3 каждая увеличивает общий leafCount. Путь состоит из локальных номеров команд: например, 1.0.1 означает выбор нити 1 на первом уровне, нити 0 во внутренней команде, затем нити 1 в следующей.

Режим	Уровень 1	Уровень 2: выполнений блока	Уровень 3: листьев
Вложенность разрешена	2 участника	2 * 2 = 4	2 * 2 * 2 = 8
Вложенность запрещена	2 участника	2 * 1 = 2	2 * 1 * 1 = 2

При запрете вложенности внутренний блок не исчезает: он исполняется сериализованно в команде из одной нити. Её локальный номер всегда 0. Поэтому пути становятся 0.0.0 и 1.0.0, а leafCount равен 2. При разрешённой вложенности наблюдаются все восемь комбинаций из нулей и единиц.

Почему счётчик защищён atomic

leafCount передаётся по ссылке и относится к одному общему объекту. Несколько команд могут одновременно дойти до leafCount++. Без atomic инкременты конфликтуют и могут потеряться. atomic защищает это обновление, critical защищает вывод; один механизм не заменяет другой.

level и parent передаются в parallel как firstprivate: участники получают корректные начальные параметры своего вызова. Переменные thread и path объявлены внутри блока и принадлежат выполняющей его нити. std::string хранит удобный путь; to_string превращает номер в текст.

Что не следует утверждать

Восемь листьев - восемь выполнений блока третьего уровня, а не «2 + 4 + 8 = 14 разных одновременно работающих физических ядер». Породившая нить входит в новую команду, а runtime управляет рабочими потоками. Номера повторяются в разных командах. Гарантия восьми листьев предполагает, что реализация действительно разрешила команды из двух нитей.

showNestedRegions() сохраняет предыдущее значение nested, показывает оба режима и восстанавливает настройку. В современных версиях OpenMP управление числом активных уровней описано через omp_set_max_active_levels(); в данном MSVC-проекте использовано API из методички.

8. Задание 5: single, master и барьеры

Свойство	single	master
Исполнитель	Одна выбранная реализацией нить команды	Только нить с номером 0 текущей команды
Сколько исполнений блока	Один раз на команду при её встрече с конструкцией	Один раз нитью-мастером, если она дошла до блока
Барьер на выходе	Есть по умолчанию	Нет
Как убрать ожидание	Опция nowait, если алгоритм допускает	Уже нет неявного ожидания
Передача приватного результата	Возможна copyprivate без nowait	Автоматической рассылки нет

Почему showSingleAndMaster() корректна

singleCount увеличивается только исполнителем single, поэтому равен 1. masterCount увеличивается только мастером и также равен 1. После single есть неявный барьер. После master поставлен явный barrier: все участники ждут завершения этапа перед выводом «завершила оба этапа». Итоговые счётчики читаются уже после parallel.

Зачем нужен барьер после записи в общий объект

```
int ready = 0;
#pragma omp parallel num_threads(4) shared(ready)
{
#pragma omp master
    ready = 1;
#pragma omp barrier
    // Теперь все могут читать ready.
}
```

Без барьера рабочая нить могла бы читать ready одновременно с записью мастера. Ожидание завершения одного защищённого действия отличается от взаимного исключения: critical допускает по одной нити внутрь участка, barrier собирает всех участников команды. Сам critical не означает «одна выбранная нить выполнит участок всего один раз»: в него по очереди могут зайти все нити.

У single нет обязательного входного барьера. Поэтому фраза «single полностью синхронизирует всё перед собой» неверна. nowait убирает выходной барьер single, но не барьер в конце всей parallel-области. Если дальнейший код использует результат single, снятие ожидания может нарушить корректность.

9. Прямые ответы на задания 6 и 7

6. Может ли нить-мастер выполнить single?

Да. Исполнитель single выбирается из участников команды, а мастер входит в эту команду. Нет правила, исключающего нить 0. В showMasterInSingle() сначала создаётся команда из одной нити: единственный возможный исполнитель - нить 0. Это простой детерминированный пример, доказывающий возможность.

Затем создаётся команда из четырёх нитей. В ней single может выполнить 0, 1, 2 или 3. Один запуск показывает только выбранного в нём исполнителя. Нельзя по этому наблюдению обещать, что при каждом следующем запуске single будет выполнять та же нить или что в нескольких последовательных single всегда будет одинаковый исполнитель.

7. Может ли нить с номером 1 выполнить master?

Нет, если речь о номере 1 в той же самой команде. Участок master исполняет только локальная нить 0. В showMasterOnly() каждая нить создаёт собственную value = 1. Мастер меняет только свою value на 2. Поэтому нить 0 выводит 2, остальные - 1. Перед выводом не нужен барьер для передачи value: никакой передачи между нитями здесь нет.

```
#pragma omp parallel num_threads(4)
{
    int value = 1;
#pragma omp master
    value = 2;
    // У нити 0: 2; у остальных: 1.
}
```

Тонкость с вложенностью: нить с номером 1 **во внешней** команде может породить внутреннюю команду и стать её нитью 0. Тогда она вправе выполнить master **внутренней** области. Это не исключение из правила: в команде, к которой относится master, её номер именно 0.

Типичная ловушка преподавателя

«Если поставить barrier после master, значение 2 появится у всех?» Нет. Барьер синхронизирует участников, но не копирует приватные переменные. Для общего результата нужен общий объект с корректным доступом либо явная рассылка, например single copyprivate.

В методичке пример master меняет локальное n сначала на 2, потом на 3. В текущем пункте 7 оставлено первое изменение: оно уже показывает нужное свойство исполнителя и независимость копий. Теоретически второй такой этап дал бы у мастера 3, у остальных снова 1.

10. Что повторить перед сдачей

Краткие вопросы с ответами

Почему get_max_threads() вернула 12, а создались 8192? Первая функция сообщает настройку команды по умолчанию, а явный num_threads задаёт другой запрос. Программные нити могут по очереди исполняться на меньшем числе процессоров.

Почему sum нельзя сделать просто shared? shared даёт общий объект, но не делает операцию += атомарной. Для суммирования используется reduction.

Почему строки идут не 0, 1, 2, 3? Планировщик и runtime не обязаны соблюдать такой порядок. critical защищает целостность вывода, не сортирует исполнителей.

Что будет при if(false)? Блок всё равно исполнится одной нитью. Это не условие пропуска вычисления и не аналог обычного if вокруг всего блока.

Есть ли неявный барьер у master? Нет. У single есть выходной барьер, если не указан допустимый nowait. В конце parallel ожидание участников всё равно есть.

Как подтверждена граница? Запрос 8192 выполнен, запрос 10000 уменьшен до 8192; уточнение привело к паре 8192/8193, и три повторные пары дали тот же результат. Следовательно, экспериментальная граница полного запроса в текущих условиях - 8192. Это не заявление о максимуме для всех устройств и реализаций OpenMP.

Какие детали C++ нужно узнать в своём коде

Анонимное пространство имён скрывает вспомогательные имена в исходном файле. const/constexpr фиксируют настройки; ссылка int& позволяет рекурсии менять один счётчик; firstprivate делает копии параметров; std::string и to_string собирают путь. cin.clear() сбрасывает ошибку ввода, ignore() удаляет плохую строку, eof() прекращает ввод, чтобы меню не зациклилось. return в ветке служебного запуска не даёт ей открыть меню.

Порядок демонстрации и готовность

1. Назвать тему и модель fork-join. 2. Показать поиск и честно прочитать его вывод. 3. Показать if для двух размеров. 4. Назвать число процессоров и оговорить отсутствие affinity. 5. Объяснить 8 и 2 листа. 6. Сравнить single/master и ответить про нить 1 во вложенной команде.

Проверки: Release/Debug x64, фактические размеры команд, суммы, вложенность, меню и EOF. Алгоритм поиска отдельно проверен на моделируемой границе 37 при ограничении команды, аварийном коде и тайм-ауте. Число 37 относится только к тесту алгоритма, не к системе. Полный текущий запуск лабораторной сохранён в output/lab2_run.txt.

Основа: предоставленная лабораторная работа №2, задания 1-7; текущие Lab2.cpp, Lab2.h и ParallelProgramming.cpp. Документ дополняет прежние пояснения актуальным экспериментом: вместо остановки на 128 получена и повторно проверена граница 8192.